LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
“Transaksi Dinamis (Polymorphism)”



disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

NURFIAH, S.ST., M.Kom.

Asisten Praktikum:

SITI FADHILAH RAHMI

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam praktikum pekan ke 5 ini, pemahaman mengenai fleksibilitas kode menjadi salah satu hal yang esensial, terutama ketika kita mulai masuk ke konsep polimorfisme (polymorphism) dan pengikatan dinamis (Dynamic Method Dispatch). Pada modul-modul sebelumnya sudah belajar bagaimana kelas turunan mewarisi sifat dari kelas induknya, seperti pada RekeningGiro. Namun, logika penarikan dananya saat itu masih menggunakan aturan standar rekening biasa yang melarang saldo bernilai negatif.

Melalui praktikum pekan 5 ini, kita belajar mengenai penerapan method overriding agar kelas anak bisa memodifikasi perilaku kelas induknya sesuai kebutuhan spesifik, seperti memberikan batas toleransi overdraft pada rekening giro. Tidak hanya berhenti di situ, keunggulan utama dari polimorfisme ini kemudian diuji melalui pembuatan fitur transfer antar rekening. Dengan memanfaatkan parameter polimorfik, sistem dapat memproses transaksi lintas jenis rekening dengan mulus tanpa perlu kerepotan mengenali tipe objek secara spesifik di awal.

1.2 Tujuan

1. Memahami secara mendalam konsep polimorfisme serta cara kerja pengikatan dinamis (Dynamic Method Dispatch) di dalam bahasa Java.
2. Menerapkan teknik method overriding untuk menyesuaikan behavior kelas induk agar sesuai dengan aturan logika pada kelas turunan, khususnya penanganan batas overdraft pada RekeningGiro.
3. Merancang metode dengan argumen polimorfik yang dapat menerima berbagai macam objek subclass (baik tabungan maupun giro) secara fleksibel, yang diimplementasikan langsung pada fitur transfer antar rekening dan diintegrasikan ke menu CLI utama.

BAB II
PEMBAHASAN

2.1 Rekening.java

```
package MiniBank.pekan5;
import java.util.ArrayList;
import java.text.NumberFormat;
import java.util.Locale;

public class Rekening {
    // Atribut privat (Enkapsulasi)
    private String nomorRekening;
    private String namaPemilik;
    private String pin; // Data sensitif

    @SuppressWarnings("deprecation")
   NumberFormat rupiah = NumberFormat.getCurrencyInstance(new Locale("id",
"ID"));

    // Gunakan protected agar subsclass bisa mengaksesnya langsung
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    // Constructor menerima input PIN dari user
    public Rekening(String nomor, String nama, double saldoAwal, String
pinInput, String pin) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;
        this.pin = pin;
        this.riwayatTransaksi = new ArrayList<>();

        // Validasi PIN 6 digit
        if (pinInput != null && pinInput.length() == 6) {
            this.pin = pinInput;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit angka!
Menggunakan PIN default 123456.");
            this.pin = "123456";
```

```
        }

        this.riwayatTransaksi = new ArrayList<>();

        // Pesan konfirmasi pembukaan rekening dan pendaftaran PIN
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat dengan saldo Rp" + saldo + " dan PIN telah terdaftar.");
    }

    // Getter
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    // Method Otentikasi PIN
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            String idTrx = "TRX-S" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo
saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening  : " + nomorRekening);
        System.out.println("Nama Pemilik  : " + namaPemilik);
        System.out.println("Saldo Akhir   : " + saldo);
    }

    public boolean tarikTunai(double nominal) {
```

```
        if (nominal <= 0) {
            System.out.println("Transaksi Gagal: Minimal nominal penarikan
harus lebih dari 0!");
            return false;
        } else if (nominal > saldo) {
         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp" + saldo);
         return false;
        } else {
         saldo -= nominal;

         String idTrx = "TRX-T" + System.currentTimeMillis();
         riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

         System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa
saldo: Rp" + saldo);
         return true; // Transaksi sukses
        }
    }

    public void cetakTotalTransaksi() {
        double totalSetor = 0;
        double totalTarik = 0;

        for (Transaksi trx : riwayatTransaksi) {
            if (trx.getJenis().equals("Kredit")) {
                totalSetor += trx.getNominal();
            } else if (trx.getJenis().equals("Debit")) {
                totalTarik += trx.getNominal();
            }
        }

        System.out.println("--- TOTAL TRANSAKSI ---");
        System.out.println("Total Setor : " + rupiah.format(totalSetor));
        System.out.println("Total Tarik : " + rupiah.format(totalTarik));

        System.out.println("\n--- 3 TRANSAKSI TERBARU ---");

        int mulai = Math.max(0, riwayatTransaksi.size() - 3);
```

```
        for (int i = mulai; i < riwayatTransaksi.size(); i++) {
            riwayatTransaksi.get(i).cetakDetail();
        }
    }

    public void transfer(Rekening tujuan, double nominal, String pinInput) {
        // Otentikasi PIN, jika PIN salah maka batalkan
        if (!otentikasi(pinInput)) {
            System.out.println("Transfer Gagal: PIN salah!");
            return;
        }

        // Validasi agar tidak transfer ke diri sendiri
        if (this.nomorRekening.equals(tujuan.getNomorRekening())) {
            System.out.println("Transfer Gagal: Tidak bisa transfer ke
rekening sendiri!");
            return;
        }

        // Disini terjadi Dynamic Dispatch, jika objeknya adalah rekening
Giro maka akan otomatis mengeksekusi tarikTunai milik Rekening Giro dengan
batas overdraft
        boolean statusTarik = this.tarikTunai(nominal);
        if (statusTarik) {
            tujuan.setorTunai(nominal);

            String idTrx = "TRX-TRF-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Transfer ke " +
tujuan.getNomorRekening(), nominal));

            System.out.println("Transfer Rp" + nominal + " ke " +
tujuan.getNomorRekening() + " berhasil!");
        }
    }
}
```

Pada class Rekening.java, kita harus merefaktor method tarikTunai. Agar fungsi penarikan dapat digunakan sebagai bagian dari fitur transfer nanti, kita perlu mengubah tipe kembalian (return type) tarikTunai dari void menjadi boolean yaitu mengembalikan true jika sukses, false jika gagal. Lebih tepatnya pada baris ke-70 sampai baris ke-86:

```
public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Transaksi Gagal: Minimal nominal penarikan
harus lebih dari 0!");
            return false;
        } else if (nominal > saldo) {
         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp" + saldo);
         return false;
        } else {
         saldo -= nominal;

         String idTrx = "TRX-T" + System.currentTimeMillis();
         riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

         System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa
saldo: Rp" + saldo);
         return true; // Transaksi sukses
        }
    }
```

Penjelasan:

- Tipe Kembalian boolean: Method ini mengembalikan nilai true (jika transaksi berhasil) atau false (jika gagal). Nilai balik ini sangat penting nanti untuk mengecek status sukses tidaknya penarikan saat fitur *transfer* dipanggil.
- Validasi Nominal (nominal <= 0): Berfungsi untuk mencegah penarikan jika pengguna memasukkan angka nol atau minus. Jika ya, program langsung mencetak pesan error dan mengembalikan false.
- Cek Saldo (nominal > saldo): Memastikan uang di rekening mencukupi. Jika nominal yang ditarik lebih besar dari saldo, transaksi ditolak dan mengembalikan false.
- Eksekusi & Pencatatan (else): Jika lolos semua validasi:
    - Saldo utama langsung dikurangi (saldo -= nominal).
    - Dibuat ID transaksi unik secara otomatis berdasarkan waktu sistem (System.currentTimeMillis()).
    - Transaksi dicatat ke dalam daftar riwayatTransaksi dengan keterangan "Debit".

- Mengembalikan nilai true sebagai tanda transaksi sukses.

2.2 RekeningGiro.java

```
package MiniBank.pekan5;

public class RekeningGiro extends Rekening {
   private double batasOverdraft;

   public RekeningGiro(String nomor, String nama, double saldoAwal, String
pinAwal, double batasOverdraft) {
         // Memanggil inisialisasi dasar dari superclass
         super(nomor, nama, saldoAwal, pinAwal, pinAwal);
         this.batasOverdraft = batasOverdraft;
   }

   // IMPLEMENTASI POLIMORFISME (OVERRIDING)
   @Override
   public boolean tarikTunai(double nominal) {
         if (nominal <= 0) {
               System.out.println("Gagal: Nominal tarik Giro harus lebih dari
0!");
               return false;
         }
         // Logika Khusus Giro: Boleh ditarik asalkan nominal tidak meebihi
(saldo + batasOverdraft)
         else if (nominal > (saldo + batasOverdraft)) {
               System.out.println("Transaksi Gagal: Melebihi batas overdraft!
Maksimal penarikan: Rp" + (saldo + batasOverdraft));
               return false;
         }
         else {
               saldo -= nominal; // Saldo bisa bernilai negatif di sini
(memakan overdraft)

               String idTrx = "TRX-G" + System.currentTimeMillis();
               riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro",
nominal));

               System.out.println("Tarik tunai Giro Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
```

```
                return true;
        }
    }

    // Getter khusus Giro
        public double getBatasOverdraft() {
                return batasOverdraft;
        }

}
```

Pada class RekeningGiro.java ini kita harus melakukan Overriding. Kita akan menimpa logika tarikTunai agar menggunakan aturan batasOverdraft. Kita harus melakukan overriding pada kelas RekeningGiro karena aturan penarikan uangnya berbeda dari rekening biasa. Rekening biasa melarang saldo bernilai minus, sedangkan rekening giro punya fitur overdraft yang mengizinkan nasabah menarik uang melebihi saldo aslinya. Dengan menimpa logika tarikTunai, Java bisa secara otomatis menjalankan aturan overdraft tersebut saat objek giro digunakan tanpa perlu mengubah struktur kode utama lainnya.

Penjelasan:

- extends Rekening: Menandakan bahwa RekeningGiro adalah kelas anak (subclass) yang mewarisi atribut dan metode dari kelas induk Rekening.
- private double batasOverdraft; Variabel khusus yang hanya dimiliki oleh rekening giro untuk menampung batas toleransi penarikan saldo minus.
- super(...): Berfungsi untuk meneruskan data inisialisasi dasar (seperti nomor, nama, saldo awal, dan PIN) ke constructor kelas induknya (Rekening).
- @Override: Anotasi penanda bahwa method tarikTunai di dalam kelas ini sengaja dibuat untuk menimpa (overriding) perilaku method tarikTunai milik kelas induk.
- Validasi Overdraft (nominal > (saldo + batasOverdraft)): Logika khusus giro yang mengecek apakah nominal penarikan melebihi total gabungan sisa saldo dan batas toleransi minus yang diizinkan.
- Saldo Negatif (saldo -= nominal): Bagian inti di mana saldo utama diperbolehkan menjadi bernilai minus karena memakan jatah batas overdraft.
- Getter getBatasOverdraft(): Method tambahan khusus untuk mengambil dan membaca nilai dari variabel batasOverdraft dari luar kelas jika diperlukan.

Output:

```
=== SISTEM PERBANKAN MINI ===

Menu Utama
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Total Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih Menu: 1
Masukkan No Rekening: 2909
Masukkan Nama Pemilik: Rara H
Masukkan Saldo Awal: 100000
Buat PIN Keamanan (6 digit): 123456

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
3. Rekening VIP
Pilih Produk: 2
Masukkan Batas Overdraft: Rp500000
Rekening atas nama Rara H berhasil dibuat dengan saldo Rp100000.0 dan PIN telah terdaftar.
Rekening berhasil dibuat!

Menu Utama
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Total Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
```

```
Pilih Menu: 3
Masukkan PIN Anda untuk verifikasi: 123456
Masukkan nominal tarik tunai: 400000
Tarik tunai Giro Rp400000.0 berhasil. Saldo saat ini: Rp-300000.0
```

2.3 Tugas 1: Fitur Transaksi Antar Rekening

Pada tugas pertama ini, kita harus menambahkan method transfer ke class Rekening.java dengan nama:

public void transfer(Rekening tujuan, double nominal, String pinInput)

- Method ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro.
- Logika Method
    - Pertama, lakukan otentikasi(pinInput). Jika false, batalkan.
    - Kedua, panggil this.tarikTunai(nominal). (overriding, jika this adalah Giro, ia otomatis menggunakan aturan overdraft).

- Ketiga, periksa kembalian nilai tarikTunai. Jika true, panggil tujuan.setorTunai(nominal).

Tugas ini terdapat pada class Rekening.java baris ke-113 sampai baris ke-136:

```
public void transfer(Rekening tujuan, double nominal, String pinInput) {
        // Otentikasi PIN, jika PIN salah maka batalkan
        if (!otentikasi(pinInput)) {
            System.out.println("Transfer Gagal: PIN salah!");
            return;
        }

        // Validasi agar tidak transfer ke diri sendiri
        if (this.nomorRekening.equals(tujuan.getNomorRekening())) {
            System.out.println("Transfer Gagal: Tidak bisa transfer ke
rekening sendiri!");
            return;
        }

        // Disini terjadi Dynamic Dispatch, jika objeknya adalah rekening
Giro maka akan otomatis mengeksekusi tarikTunai milik Rekening Giro dengan
batas overdraft
        boolean statusTarik = this.tarikTunai(nominal);
        if (statusTarik) {
            tujuan.setorTunai(nominal);

            String idTrx = "TRX-TRF-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Transfer ke " +
tujuan.getNomorRekening(), nominal));

            System.out.println("Transfer Rp" + nominal + " ke " +
tujuan.getNomorRekening() + " berhasil!");
        }
    }
```

Penjelasan:

- Parameter Polimorfik (Rekening tujuan): Method ini menerima objek bertipe kelas induk (Rekening), sehingga bisa digunakan secara fleksibel untuk menerima berbagai jenis rekening turunan (baik Tabungan maupun Giro).

- Otentikasi PIN (!otentikasi(pinInput)): Berfungsi sebagai pengaman awal. Jika PIN yang dimasukkan salah, proses transfer langsung dibatalkan menggunakan perintah return.
- Validasi Rekening Sendiri: Mengecek apakah nomor rekening pengirim sama dengan nomor rekening tujuan. Jika sama, transaksi ditolak untuk mencegah pengguna mentransfer uang ke rekeningnya sendiri.
- Eksekusi *Dynamic Dispatch* (this.tarikTunai(nominal)): Bagian penting di mana method tarikTunai dipanggil. Karena sifat polimorfisme, jika akun pengirim adalah Giro, Java akan otomatis menjalankan aturan penarikan milik Giro (termasuk cek *overdraft*) tanpa perlu struktur if-else tambahan.
- Penyetoran ke Tujuan (tujuan.setorTunai(nominal)): Dijalankan hanya jika proses penarikan saldo pengirim bernilai sukses (statusTarik bernilai true), di mana uang tersebut langsung ditambahkan ke rekening tujuan.
- Pencatatan Riwayat Transaksi: Membuat ID unik berdasarkan waktu sistem (System.currentTimeMillis()) lalu merekam aktivitas transfer keluar ke dalam daftar riwayatTransaksi milik akun pengirim.

2.4 Tugas 2: Integrasi ke CLI Utama

Pada tugas 2 ini, kita mengubah menu 8 yang ada di class Main.java

- Minta pengguna memasukkan Nomor Rekening Tujuan.
- Cari nomor tersebut di dalam daftarRekening.
- Jika ketemu, minta nominal transfer dan PIN, lalu panggil akunAktif.transfer(akunTujuan, nominal, pin).

```
System.out.println("\nMenu Utama");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("6. Total Transaksi");
            System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
            System.out.println("8. Transfer ke Rekening Lain");
            System.out.println("9. Keluar");
            System.out.print("Pilih Menu: ");
```

Lalu lanjut blok case 8 dengan logika Transfer Antar Rekening sebagai berikut:

```
   case 8:
                        System.out.println("\n--- MENU TRANSFER ANTAR REKENING
---");
                        if (akunAktif == null) {
                                System.out.println("Error: Anda belum memiliki
rekening atau belum memilih akun aktif!");
                                break;
                        }

                        if (daftarRekening.size() < 2) {
                                System.out.println("Transfer Gagal: Minimal harus
ada 2 rekening terdaftar di sistem untuk melakukan transfer!");
                                break;
                        }

                        System.out.print("Masukkan Nomor Rekening Tujuan: ");
                        String noRekTujuan = input.nextLine();

                        // Cek agar tidak mentransfer ke diri sendiri
                        if (akunAktif.getNomorRekening().equals(noRekTujuan)) {
                                System.out.println("Transfer Gagal: Tidak dapat
mentransfer ke rekening sendiri!");
                                break;
                        }

                        // cari rekening tujuan di dalam daftarRekening
                        Rekening akunTujuan = null;
                        for (Rekening r : daftarRekening) {
                                if (r.getNomorRekening().equals(noRekTujuan)) {
                                        akunTujuan = r;
                                        break;
                                }
                        }

                        if (akunTujuan == null) {
                                System.out.println("Transfer Gagal: Rekening
tujuan tidak ditemukan!");
                        } else {
                                System.out.print("Masukkan nominal transfer:
Rp");
```

```
                            double nominalTransfer = input.nextDouble();
                            input.nextLine();

                            System.out.print("Masukkan PIN Anda untuk
Verifikasi: ");
                            String pinKonfirmasi = input.nextLine();

                            // Memanggil method transfer poliformik di
Rekening.java
                            akunAktif.transfer(akunTujuan, nominalTransfer,
pinKonfirmasi);
                        }
                    break;
```

Penjelasan:

- Validasi Akun Aktif & Jumlah Rekening: Program mengecek apakah sudah ada akun yang dipilih (akunAktif != null) dan memastikan total rekening terdaftar di daftarRekening minimal ada 2 buah agar aktivitas transfer bisa dilakukan.
- Pencegahan Transfer Mandiri: Terdapat validasi untuk memastikan nomor rekening tujuan tidak sama dengan nomor rekening pengirim (akunAktif.getNomorRekening().equals(noRekTujuan)).
- Pencarian Rekening Tujuan (for-each loop): Sistem menelusuri seluruh isi daftarRekening untuk mencocokkan nomor rekening yang diinput pengguna dengan data yang ada di memori, lalu menyimpannya ke variabel penampung akunTujuan.
- Eksekusi Transfer Polimorfik: Setelah nominal dan PIN verifikasi diinput oleh pengguna, sistem langsung memanggil method akunAktif.transfer(akunTujuan, nominalTransfer, pinKonfirmasi) yang memanfaatkan konsep polimorfisme.

Output:

```
=== SISTEM PERBANKAN MINI ===

Menu Utama
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Total Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih Menu: 1
Masukkan No Rekening: 2909
Masukkan Nama Pemilik: Rara H
Masukkan Saldo Awal: 350000
Buat PIN Keamanan (6 digit): 123456

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
3. Rekening VIP
Pilih Produk: 1
Masukkan Suku Bunga (%): 30
Rekening atas nama Rara H berhasil dibuat dengan saldo Rp350000.0 dan PIN telah terdaftar.
Rekening berhasil dibuat!

Menu Utama
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Total Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
```

```
Pilih Menu: 1
Masukkan No Rekening: 0909
Masukkan Nama Pemilik: Naira R
Masukkan Saldo Awal: 500000
Buat PIN Keamanan (6 digit): 123456

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
3. Rekening VIP
Pilih Produk: 1
Masukkan Suku Bunga (%): 50
Rekening atas nama Naira R berhasil dibuat dengan saldo Rp500000.0 dan PIN telah terdaftar
Rekening berhasil dibuat!

Menu Utama
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Total Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
9. Keluar
Pilih Menu: 5
Masukkan nomor rekening: 2909
Akun berhasil diganti ke Rara H

Menu Utama
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Total Transaksi
7. Simulasi Akhir Bulan (Khusus Tabungan)
```

```
8. Transfer ke Rekening Lain
9. Keluar
Pilih Menu: 8

--- MENU TRANSFER ANTAR REKENING ---
Masukkan Nomor Rekening Tujuan: 0909
Masukkan nominal transfer: Rp150000
Masukkan PIN Anda untuk Verifikasi: 123456
Tarik tunai Rp150000.0 berhasil. Sisa saldo: Rp200000.0
Setor tunai Rp150000.0 berhasil. Saldo saat ini: Rp650000.0
Transfer Rp150000.0 ke 0909 berhasil!
```

BAB III
REVIEW

3.1 Praktikum 1: Class, Object, Constructor dan Method (Interaksi Objek & Manipulasi State (Mini Bank))

Pada pekan pertama, kita mempelajari fondasi dasar pemrograman berorientasi objek dengan memodelkan entitas dunia nyata ke dalam bentuk class sebagai cetak biru (blueprint) dan menginstansiasikannya menjadi sebuah object. Kita belajar menyusun atribut dasar seperti nomor rekening, nama pemilik, dan saldo, serta membuat constructor untuk menginisialisasi state awal tersebut. Selain itu, interaksi program dirancang menggunakan antarmuka berbasis konsol (CLI) dengan memanfaatkan Scanner dan while-loop.

3.2 Praktikum 2: Relasi Antar Objek (Class Diagram & Asosiasi)

Modul praktikum Pemrograman Berorientasi Objek ini berfokus pada Relasi Antar Objek (Class Diagram & Asosiasi). Praktikum ini bertujuan agar mahasiswa mampu membaca serta menerjemahkan class diagram ke dalam kode program Java, memahami dan mengimplementasikan relasi asosiasi satu ke banyak (one-to-many), serta memanajemen koleksi objek di dalam objek lain menggunakan struktur data dinamis seperti ArrayList. Melalui studi kasus perbankan, sistem dikembangkan agar setiap kali terjadi aktivitas penyetoran atau penarikan tunai, sistem akan secara otomatis membuat objek Transaksi dan menyimpannya ke dalam riwayat transaksi pada objek Rekening.

3.3 Praktikum 3: Keamanan & Integritas Data (Enkapsulasi)

Modul praktikum Pemrograman Berorientasi Objek ini berfokus pada konsep keamanan dan integritas data melalui Enkapsulasi (Information Hiding). Praktikum ini bertujuan agar kita memahami pentingnya menyembunyikan detail implementasi internal, mengimplementasikan access modifiers (private, public, protected) dengan tepat, mengontrol akses pembacaan serta penulisan atribut menggunakan Getter dan Setter, serta menerapkan validasi business logic saat proses autentikasi PIN. Melalui studi kasus perbankan, atribut kelas seperti Transaksi dan Rekening dikunci menggunakan modifier private agar terhindar dari manipulasi data secara langsung dari luar kelas, serta dilengkapi dengan metode autentikasi PIN dan aturan panjang karakter yang valid.

3.4 Praktikum 4: Ekspansi Produk Keuangan (Inheritance)

Modul praktikum Pemrograman Berorientasi Objek ini membahas tentang penerapan konsep Inheritance (pewarisan) dalam bahasa pemrograman Java untuk memperluas sistem produk keuangan. Praktikum ini bertujuan agar kita memahami prinsip Code Reusability, mampu membedakan superclass dan subclass, menggunakan kata kunci extends serta super(), serta menerapkan konsep upcasting dan downcasting pada sekumpulan objek. Melalui studi kasus perbankan, kelas utama Rekening direfaktor menjadi superclass, kemudian diturunkan menjadi produk spesifik yaitu RekeningTabungan dan RekeningGiro guna menghindari duplikasi kode dan menerapkan prinsip Don't Repeat Yourself (DRY).

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan pelaksanaan praktikum ini, dapat disimpulkan bahwa penerapan konsep polimorfisme dan Dynamic Method Dispatch memberikan fleksibilitas yang sangat membantu dalam perancangan program berorientasi objek. Melalui teknik method overriding pada kelas RekeningGiro, kita bisa memodifikasi aturan penarikan dana agar mengakomodasi batas toleransi saldo negatif (overdraft) tanpa harus merusak struktur dasar dari kelas induknya. Selain itu, penggunaan parameter polimorfik pada fitur transfer membuktikan bahwa sistem mampu memproses transaksi lintas jenis rekening secara mulus dan dinamis. Seluruh rangkaian logika tersebut juga berhasil diintegrasikan dengan baik ke dalam menu aplikasi perbankan berbasis CLI, sehingga menghasilkan program yang jauh lebih rapi, terstruktur, dan fungsional.

LINK REPOSITORY GITHUB:

https://github.com/nairarama6/PrakPBO2026_B_2511533027/tree/Pekan5/PrakPBO2026

LINK VIDEO:

https://youtu.be/FS-ZqjbCvew